

Desenvolvimento Android

MOV786205 – CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br



INSTITUTO FEDERAL
Santa Catarina
Campus São José

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

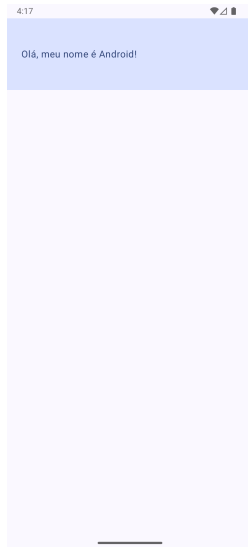
Assuntos da aula

- 1 Como criar um projeto com Android Studio
- 2 Como criar um aplicativo Android simples com *Jetpack Compose*
- 3 Como visualizar um aplicativo na tela de *Preview Jetpack Compose*
- 4 Como executar um aplicativo no emulador Android
- 5 Como executar um aplicativo em um dispositivo Android
- 6 Criar diferentes previews para diferentes composições de tela

Aplicativo a ser desenvolvido

Olá, meu nome é Android!

- Acima a tela no Preview do *Jetpack Compose*
- Ao lado a tela do aplicativo no emulador Android



Código da função que gera a tela do aplicativo

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Surface(
        color = MaterialTheme.colorScheme.primaryContainer,
        modifier = modifier.fillMaxWidth()
    ) {
        Text(
            text = "Olá, meu nome é $name!",
            fontSize = 16.sp,
            modifier = Modifier.padding(24.dp)
        )
    }
}
```

■ <https://developer.android.com/develop/ui/compose/documentation>

Preview de composições

- O Preview do Jetpack Compose permite visualizar diferentes estados da UI de forma rápida e prática.
- É possível criar múltiplas previews para uma mesma função composável, facilitando a comparação entre diferentes layouts e estados.

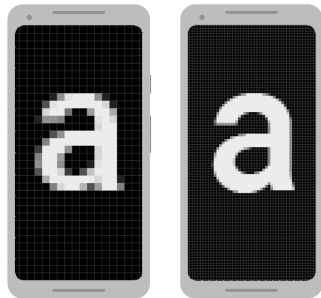
```
// Preview de uma função composável
@Preview
@Composable
fun GreetingPreview() {
    Greeting(name = "Android")
}

// Com background e dimensões específicas e imitando a interface do Android
@Preview(showBackground = true, widthDp = 320, showSystemUi = true)
@Composable
fun GreetingPreviewSmall() {
    Greeting(name = "Android")
}
```

Suporte a diferentes densidades de pixel

Permite adaptar o layout a diferentes tamanhos e resoluções de tela

- **Density-independent Pixels (dp)** são usados para definir tamanhos de layout e espaçamento
- **Scale-independent Pixels (sp)** são usados para definir tamanhos de fonte
- Recursos gráficos (imagens, ícones) devem ser fornecidos em diferentes resoluções
 - mdpi (1x), hdpi (1.5x), xhdpi (2x), xxhdpi (3x), xxxhdpi (4x)
- Recursos vetoriais são recomendados para ícones e imagens que precisam ser escaláveis
 - SVG devem ser convertidos para o formato *Vector Drawable* do Android



Duas telas de mesmo tamanho físico, mas com diferentes densidades de pixel

Fonte: (Google, 2025)

Aula baseada em



GOOGLE. **Android Developer**. 2025. Disponível em: <https://developer.android.com/>.
Acesso em: 9 jul. 2025.